UNIVERSIDAD AUTONÓMA DEL ESTADO DE MÉXICO

Centro Universitario UAEM Nezahualcóyotl

Carrera:

Ingeniería en Sistemas Inteligentes

Proyecto Lenguaje de Señas

profesor Víctor Antonio Flores Lezama

EQUIPO:
Meneses Atzin Lesly Guadalupe
Velázquez Juárez Aixa Michelle
Venegas Alvarado Aranza Citlami

ISI TURNO: MATUTINO

SEMESTRE noveno

GRUPO “S9”

índice

Introducción 3
Contenido 3
Ui (interfaz de usuario) 5
Herramientas y materiales 6
Repositorio (Github ejem) 7
Estado del arte 13
Desarrollo de su software o App 13
Big data (si no lo tiene especifique que uso) 15
Capturas beta 16
UX(Experiencia del usuario) 17
Referencias biográficas 18
Resumen del proyecto 19

Introducción

La comunicación es una necesidad fundamental del ser humano y un derecho básico para la participación plena en la sociedad. Las personas con discapacidad auditiva enfrentan barreras significativas para interactuar con la comunidad oyente, que en su mayoría desconoce su lengua. Según la Organización Mundial de la Salud (OMS), más del 5 % de la población mundial, equivalente a 360 millones de personas, padece pérdida de la audición (Guerrero-Balaguera & Pérez-Holguín, 2015). En México, la mayoría de las personas desconoce la Lengua de Señas Mexicana (LSM), lo que limita el desarrollo educativo, profesional y social de esta comunidad (Mejía Pérez & Córdova Esparza, 2023).

La lengua de señas es el principal medio de comunicación de las personas sordas. A diferencia de las lenguas orales, utiliza el canal viso-gestual en lugar del audio-vocal (Tovar, 2001) y posee una estructura lingüística propia, basada en parámetros como la configuración de la mano, el movimiento, la ubicación, la orientación y los rasgos no manuales (Rodríguez-González, 1992; Mejía Pérez & Córdova Esparza, 2023). Cada país tiene su propio lenguaje de señas, como la LSM, la Lengua de Señas Colombiana (LSC) y la Lengua de Señas Americana (ASL) (Guerrero-Balaguera & Pérez-Holguín, 2015). Aunque permite la comunicación entre personas sordas, no facilita la interacción con quienes no la conocen, lo que genera aislamiento y exclusión. La presencia de una persona que sepa la lengua de señas suele ser necesaria, pero no siempre está disponible, lo que restringe la autonomía de esta población en la educación, la salud, el trabajo y los servicios públicos.

Ante este problema, los sistemas automáticos de reconocimiento de lengua de señas son una alternativa tecnológica viable. Estos sistemas emplean visión por computadora y aprendizaje automático para identificar las señas y traducirlas a texto o voz, sin dispositivos invasivos ni intérpretes humanos (Mejía Pérez & Córdova Esparza, 2023). En este campo, el aprendizaje profundo ha demostrado ser muy efectivo en el reconocimiento de patrones y la clasificación de objetos. Las redes neuronales artificiales, inspiradas en las biológicas, se estructuran en capas de entrada, ocultas y de salida; entre las más usadas están las redes neuronales profundas (DNN), las convolucionales (CNN), ideales para imágenes, y las recurrentes (RNN), que clasifican datos que varían en el tiempo, como secuencias de movimiento (Mejía Pérez & Córdova Esparza, 2023).

Contenido

¿Qué es?

Es un sistema de visión por computadora que reconoce en tiempo real las letras del abecedario de la Lengua de Señas Mexicana (LSM) a través de una cámara web y las convierte en texto en pantalla, lo que permite formar palabras letra por letra.

¿Cómo funciona?

1. La cámara web captura el video en tiempo real (OpenCV).
2. MediaPipe Hands detecta la mano y extrae 21 puntos clave (63 valores: x, y, z).
3. Los puntos se normalizan (se toma la muñeca como origen y se escalan).
4. La red neuronal MLP clasifica la letra y entrega su nivel de confianza.
5. La interfaz muestra la letra, la confianza y la palabra que se va formando.

Alcance

- Reconocimiento de letras estáticas del alfabeto dactilológico (A-Z + Ñ).
- Detección de una sola mano.
- Captura y predicción en tiempo real.
- Formación de palabras letra por letra.
- Modo de recolección de datos para grabar muestras propias.
- Inicio de sesión de usuarios.

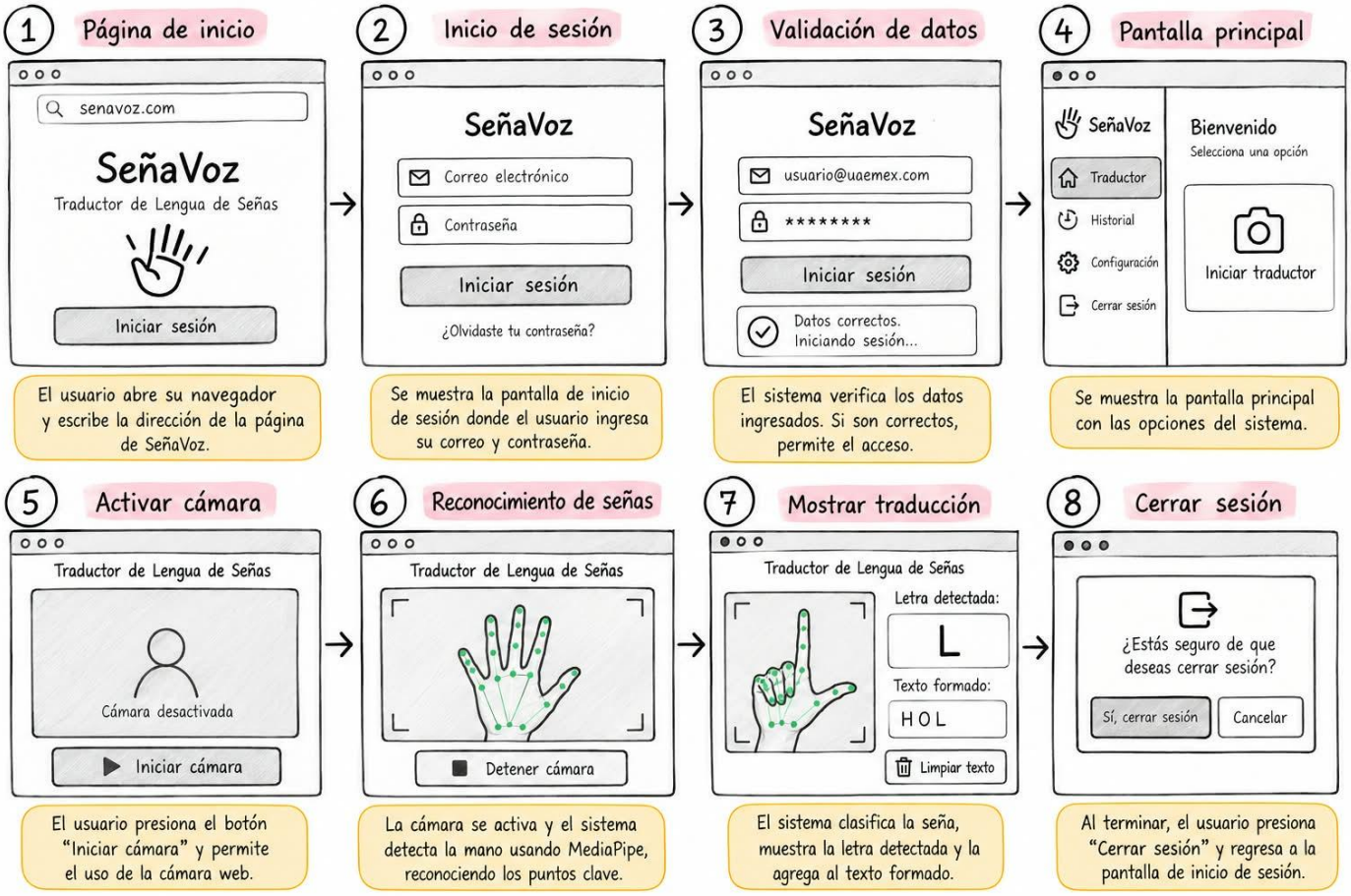
Fuera del alcance

- Text-to-speech (el botón "Reproducir en voz alta" queda deshabilitado; se contempla como trabajo futuro).
- Señas dinámicas completas (J y Z con movimiento).
- Frases y oraciones completas.
- Diccionario o autocompletado de palabras.
- Corrección gramatical.
- Reconocimiento de números (0-9).
- Dos manos simultáneas.
- Expresiones faciales y lenguaje corporal.
- Traducción inversa (texto o voz a señas).

Arquitectura en capas

Capa	Contenido
Presentación	Login e interfaz del traductor (Streamlit).
Lógica	Detector de mano (MediaPipe) y modelo entrenado (MLP).
Datos	Base de datos de usuarios y dataset de landmarks (CSV).

Ui (interfaz de usuario)



Herramientas y materiales

Herramienta	**Uso**
Python 3.11	Lenguaje principal.
Visual Studio Code	Editor de código.
Docker / docker-compose	Contenedor y despliegue reproducible.
Git y GitHub	Control de versiones.

Librerías

Librería	**Uso**
OpenCV	Captura de video.
MediaPipe	Detección de la mano y extracción de 21 landmarks.
NumPy	Operaciones con arreglos.
TensorFlow / Keras	Construcción y entrenamiento de la red neuronal.
Scikit-learn	Métricas: matriz de confusión, accuracy, classification report.
Pandas	Manejo del CSV de landmarks y etiquetas.
Joblib	Guardado del modelo y las etiquetas.
Streamlit	Interfaz gráfica.
streamlit-webrtc	Cámara en el navegador (útil con Docker).
SQLite + SQLAlchemy	Base de datos de usuarios.
bcrypt	Cifrado de contraseñas.
python-dotenv	Variables de entorno (sin datos escritos directamente en el código).
Pillow	Manejo de imágenes en la interfaz.
Matplotlib / Seaborn	Gráficas de resultados.

Calidad y pruebas

- flake8 y pylint (estilo), black (formato), pytest (pruebas).

Diseño y documentación

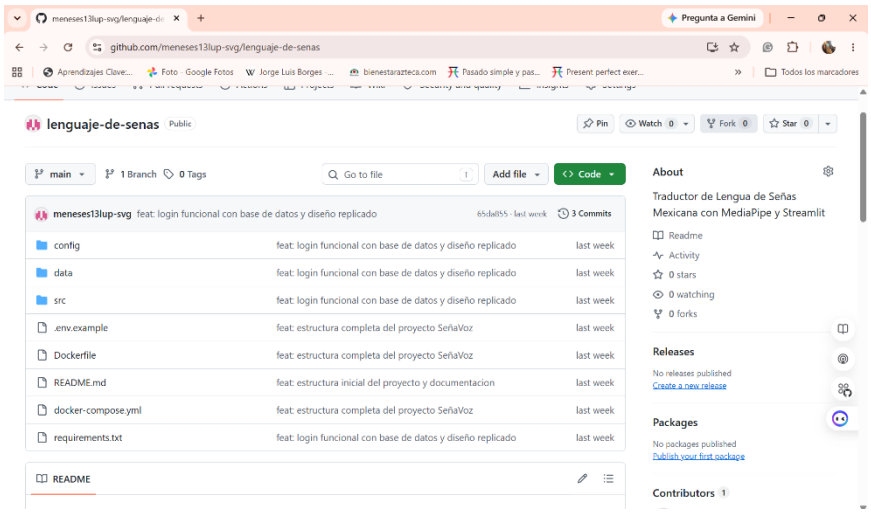
- Figma o Excalidraw (interfaces), draw.io o Mermaid (diagramas), Zotero (referencias).

Materiales

- Computadora sin GPU dedicada.
- Cámara web estándar.
- Buena iluminación frontal y fondo uniforme.
- Alfabeto oficial de LSM como referencia.
- Voluntarios para el dataset, con formato de consentimiento.
- Si es posible, validación con una persona sorda o un intérprete de LSM.

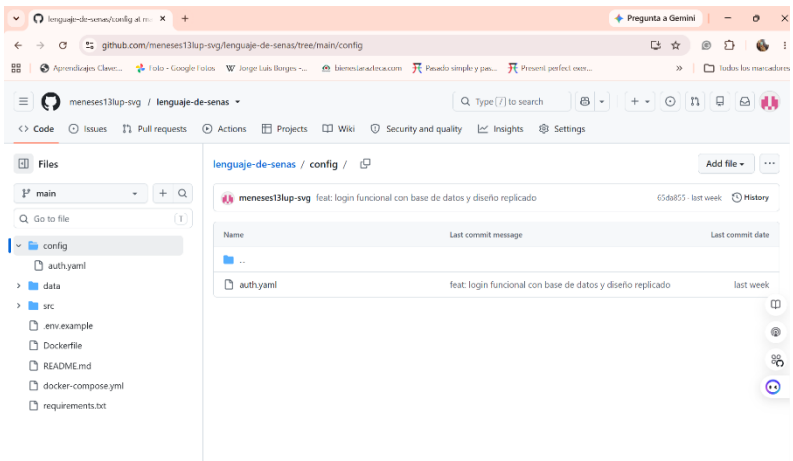
Repositorio (Github ejem):

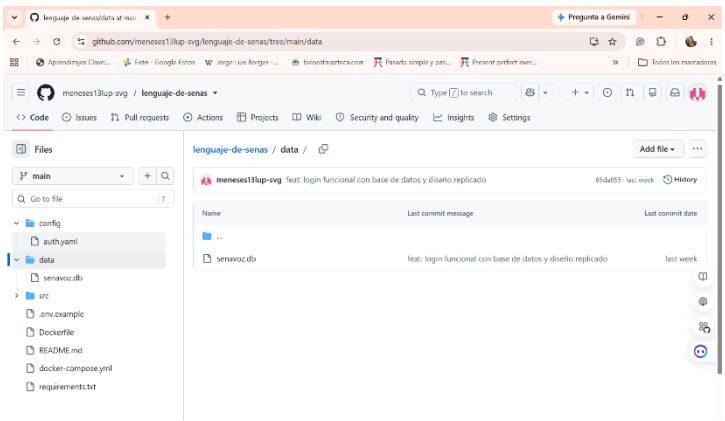
En este apartado se mostrará el repositorio con el que se está trabajando.
En este apartado se va a poner el link en donde puedes encontrar nuestro trabajo en GitHub
https://github.com/meneses13lup-svg/lenguaje-de-senas



Aquí se muestran las carpetas con las que se trabaja: config, data, src, .env, example, README.md, Docker, requerements.txt

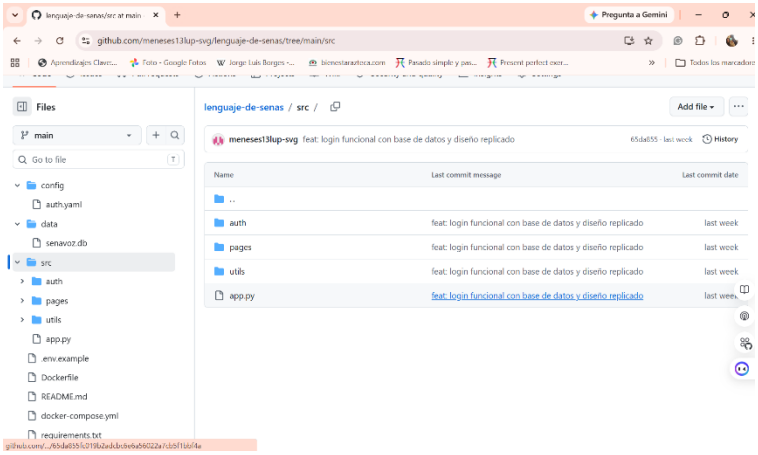
- CARPETA config: en esta carpeta están los archivos de auth.yaml donde se guardan las credenciales de los usuarios cabe aclarar que esta carpeta no se sube ya que contiene cosas y es por seguridad se excluye.

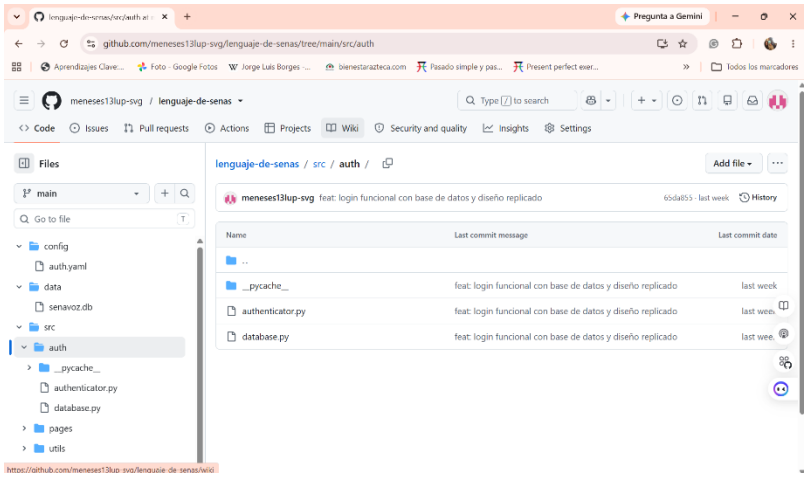




- CARPETA data: aquí se va a almacenar la base de datos donde se van a guardar los usuarios registrados y el historial, así como los dataset de puntos con la que se va a entrenar la red neuronal.

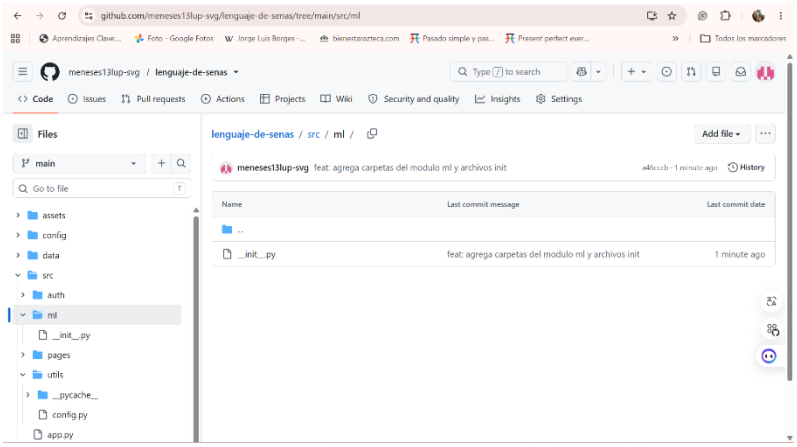
- CARPETA src: esta carpeta contine todo el software donde vamos a poner el código y esta carpeta se divide en auth, ml, pages, utils y app.py.



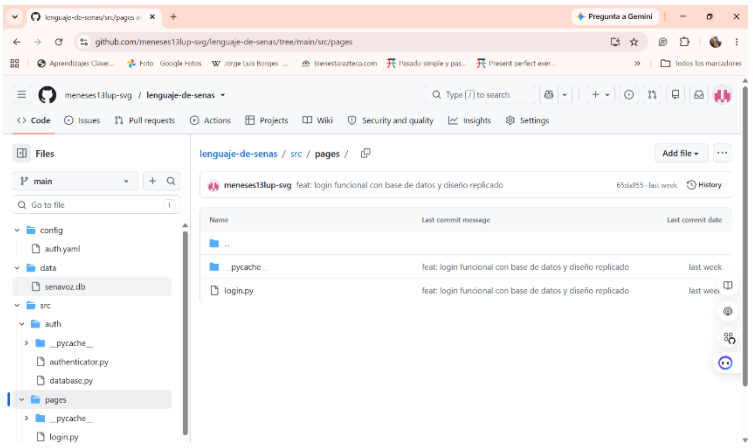


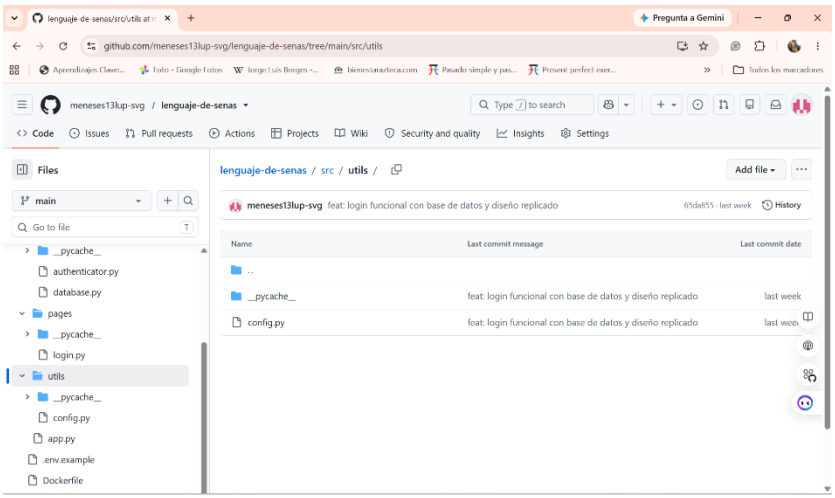
- La carpeta auth es para la seguridad es decir para la validación de contraseñas, conexión con la base de datos.

- La carpeta ml es para el código de IA, es decir, para la detección de manos, el entrenamiento y la predicción.



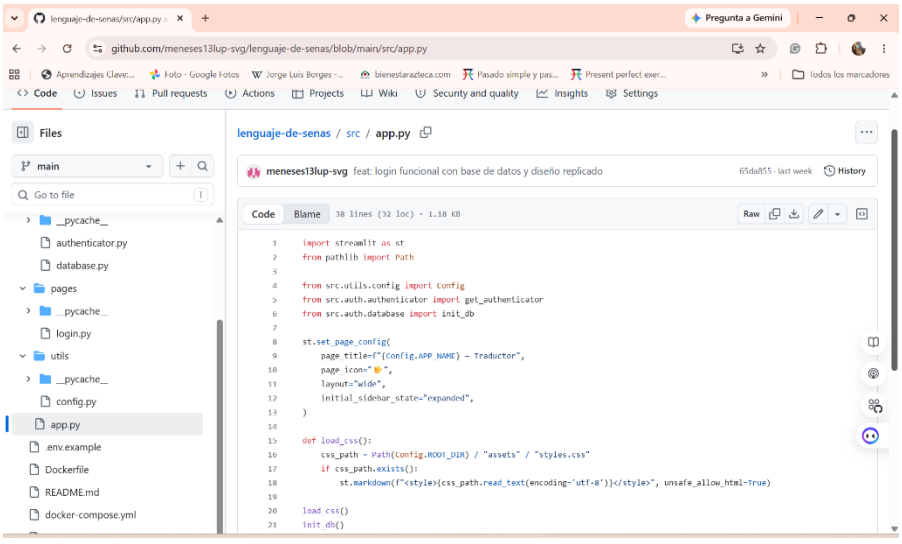
- La carpeta pages está destinada para la interfaz gráfica para el diseño del login, la pantalla del traductor entre otras cosas.



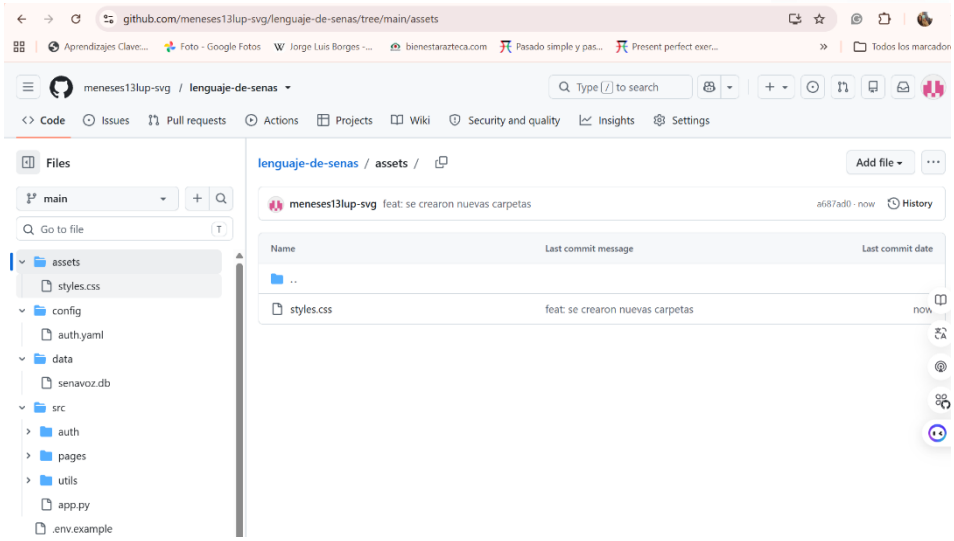


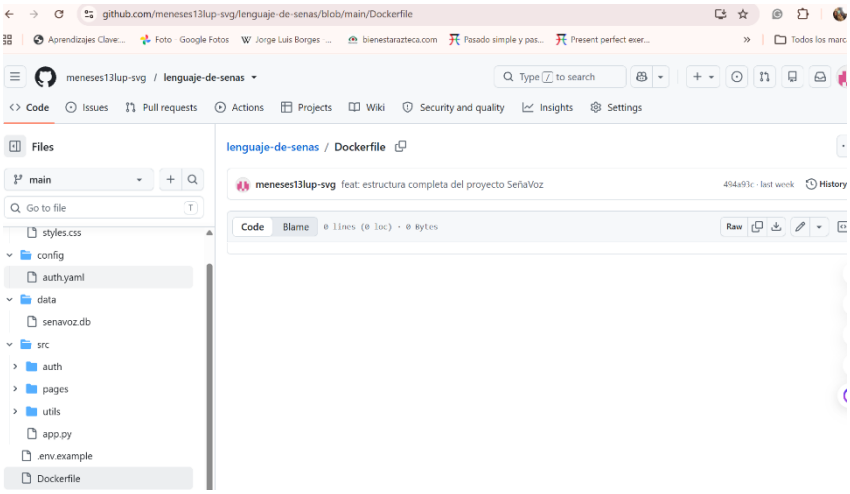
- La carpeta utils tiene las funciones de apoyo como para cargar las variables de entorno.

- El archivo app.py tiene la función de analizar la aplicación.



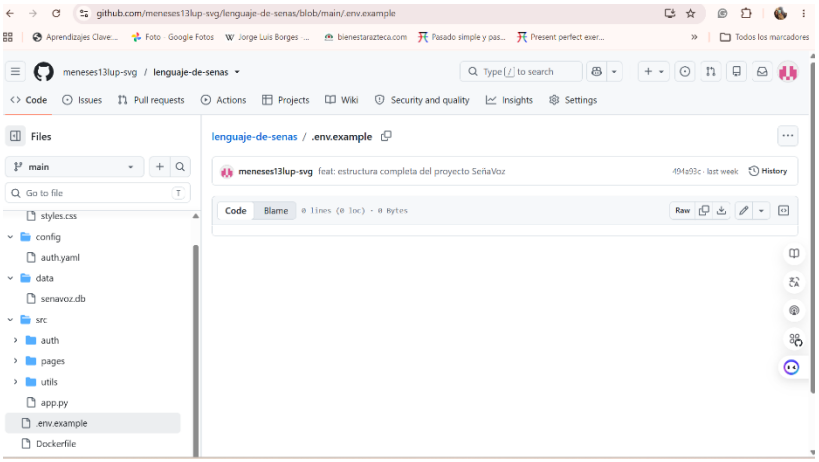
- CARPETA assets: este contiene los archivos estáticos como el diseño para que la interfaz no se vea mal.



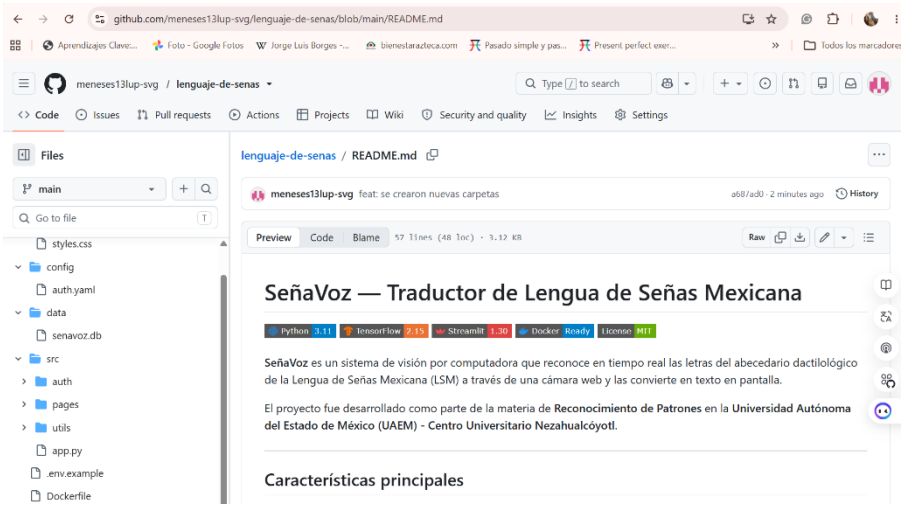


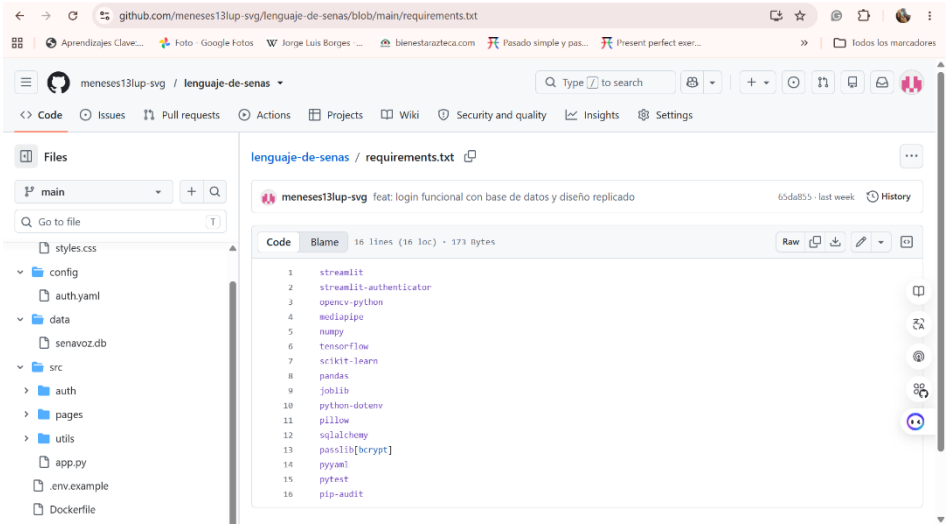
- Dockerfile: Guarda los archivos que contienen las instrucciones de empaquetar el software y pueda correr en cualquier computadora.

- .evn: Archivo que muestra que variables de entorno necesita el sistema para funcionar.

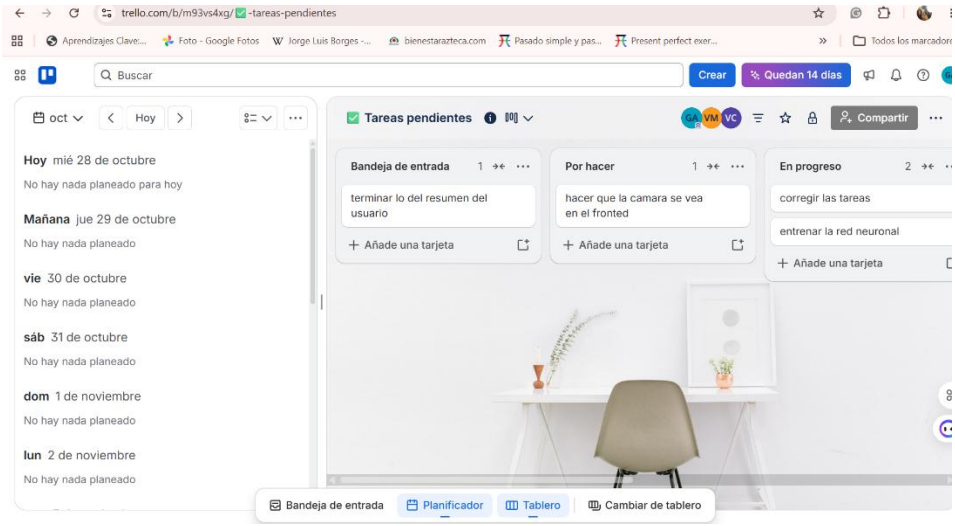


- README.md: es la portada del repositorio que se ve en github



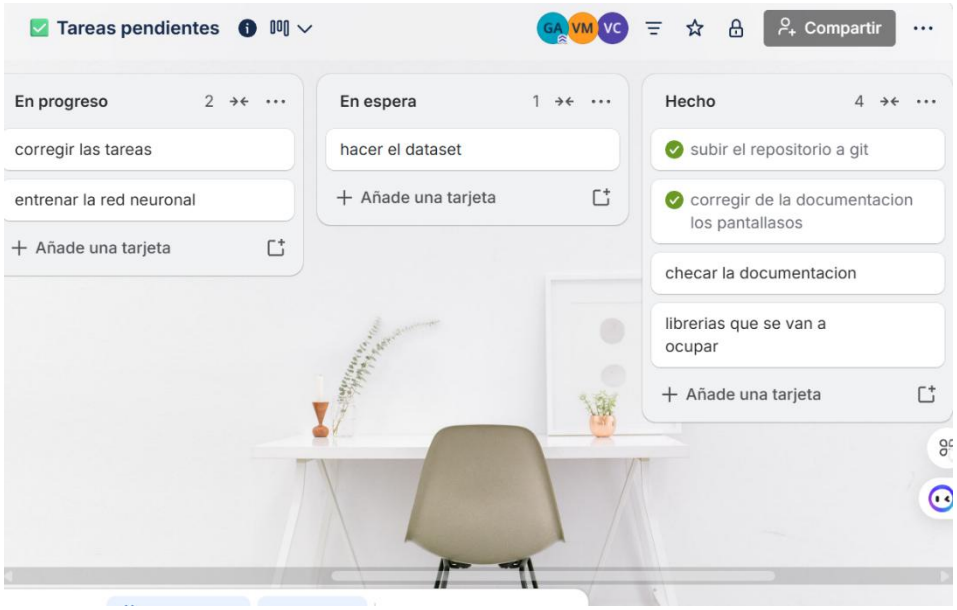


- Requierements.txt: en este contiene la lista exacta de las librerías que se ocupan junto con las versiones que el proyecto necesita para ejecutarse.



En esta captura de pantalla se pueden ver los trbajams que hemos estado haciendo para llevar a cabo el proyecto

Aquí se muestran algunas de las tareas que aun estan en espera y otras que ya se hicieron y en espera esta el hacer el dataset ya que es el mas laboreoso y el que mejor se debe de hacer porque es con el que se va a entrenar la red neuronal



Estado del arte

1. Enfoques generales

El reconocimiento automático de la lengua de señas se ha abordado con guantes instrumentados y con visión por computadora (Guerrero-Balaguera & Pérez-Holguín, 2015). Los guantes exigen dispositivos especiales que limitan la naturalidad de la comunicación; las cámaras permiten una interacción más libre. Las etapas típicas son captura, preprocesamiento, extracción de características y reconocimiento.

2. Trabajos previos en LSM y LSC

LSM con RNN y cámara de profundidad. Mejía Pérez y Córdova Esparza (2023) reconocieron el abecedario dactilológico de la LSM con una cámara OAK-D y redes neuronales recurrentes. Usaron 27 señas (21 estáticas y 6 dinámicas) y extrajeron con MediaPipe 201 columnas de coordenadas por fotograma. Su mejor configuración (32-64 neuronas) alcanzó 80.74 % de exactitud. Las señas más confundidas fueron C con W y T con S, por los rasgos que comparten. Su limitación es el costo de la cámara de profundidad y una exactitud mejorable.

LSC con MLP en FPGA. Guerrero-Balaguera y Pérez-Holguín (2015) desarrollaron un traductor de la Lengua de Señas Colombiana a texto en una FPGA, para 23 señas estáticas. Segmentaron la mano por umbral, extrajeron vectores de proyección y clasificaron con un perceptrón multicapa de 60 neuronas ocultas, con una tasa de reconocimiento de 98.15 %. Sus limitaciones son la necesidad de fondo negro e iluminación constante, y el alto costo del hardware.

3. Trabajos con MediaPipe

Extraer landmarks con MediaPipe y clasificarlos con una CNN, un MLP o una LSTM es un enfoque eficiente. En ASL se reportan precisiones superiores al 98 % con 63 características (21 landmarks × 3 coordenadas) y una CNN compacta (Modi et al., 2026); en lengua de señas coreana, una LSTM alcanzó 94.82 % (KCI, 2026). Esto indica que no se requiere cámara de profundidad.

4. Justificación del enfoque propuesto

MediaPipe es un estándar por su eficiencia y precisión (Zhang et al., 2020), y un MLP basta para clasificar señas estáticas con resultados superiores al 90 %. Sin embargo, hay pocos sistemas accesibles para LSM de bajo costo. Este proyecto propone un traductor de letras para LSM que usa MediaPipe Hands (63 características), un MLP, una cámara web estándar sin GPU, despliegue en Docker y una meta de al menos 90 % de precisión en tiempo real.

Desarrollo de su software o App

Lenguaje de programación y paradigma a ocupar:
El proyecto se desarrolla íntegramente en Python 3.11, elegido por las siguientes razones:

- Ecosistema de machine learning: las librerías líderes (TensorFlow, scikit-learn) y de visión por computadora (OpenCV, MediaPipe) están maduras en Python.
- Curva de aprendizaje: los tres integrantes del equipo ya tienen experiencia en Python.
- Integración con Streamlit: el framework de interfaz gráfica elegido funciona de forma nativa con Python.
- Comunidad y documentación: abundante soporte para resolver dudas técnicas.

Paradigma: programación orientada a objetos (POO) y modular, con separación clara de responsabilidades:

- Módulos de captura (cámara + MediaPipe).
- Módulos de ML (recolección, entrenamiento, predicción).
- Módulos de autenticación y base de datos.
- Módulos de interfaz (Streamlit).

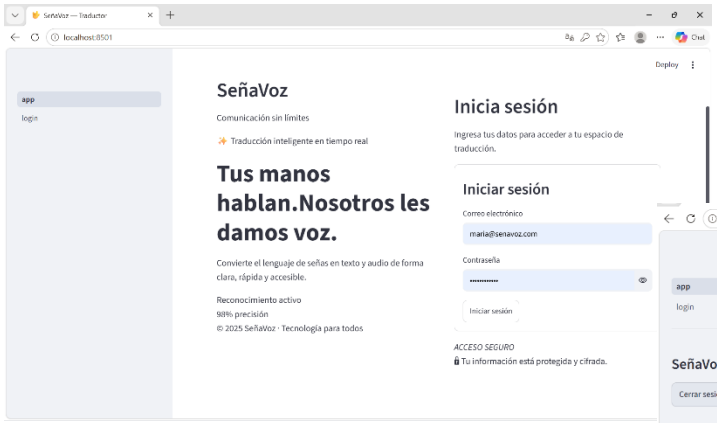
Estilo de código: PEP 8 (estándar oficial de Python), verificado con flake8 y pylint.
Al igual se ocupará visual studio code
Arquitectura que se tiene pensado en el sistema: el sistema sigue una arquitectura en capas para garantizar muchas cosas
En la primera capa va la capa de presentación es decir se vería el login en el que el usuario pueda entrar junto con el traductor
En la segunda capa que es la capa de la lógica iría el modelo entrenado junto con el detector
En la última capa seria la capa de datos donde llevaría lo de la base de datos y el dataset
¿Por qué usar Docker?
Docker empaqueta es decir es un contenedor que ayuda a que el programa funcione en cualquier maquina y esto igual ayuda a que las dependencias del proyecto no choquen con las del sistema e igual aquí nos presentamos con un problema ya que docker no tiene acceso a la cámara del hots por defecto.

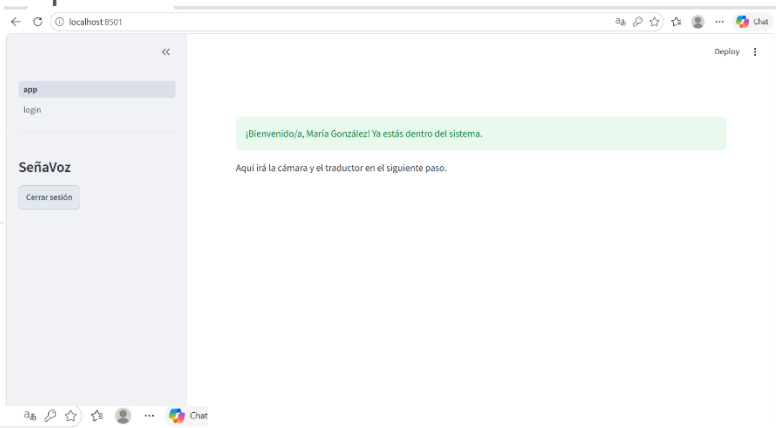
Big data

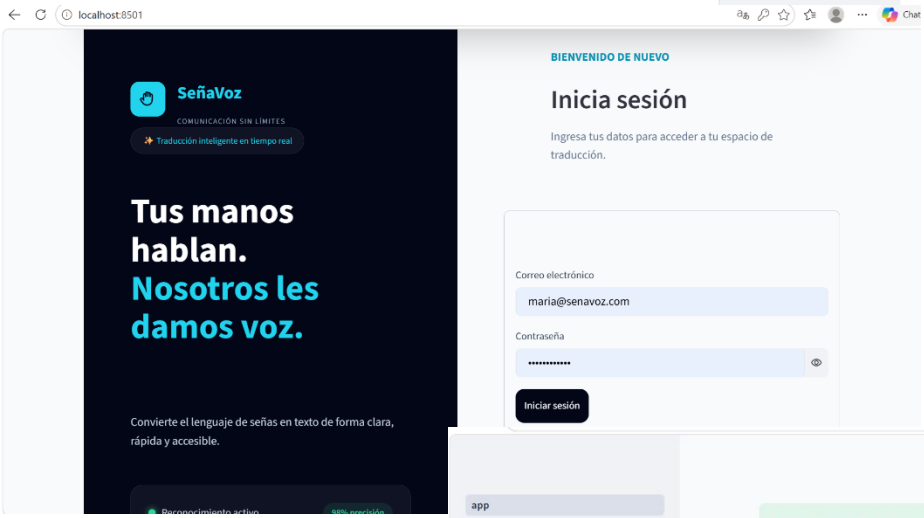
En este proyecto no se ocupa big data ya que en el proyecto que estamos llevando a cabo solo usamos machine learning (aprendizaje automático) y reconocimiento de patrones

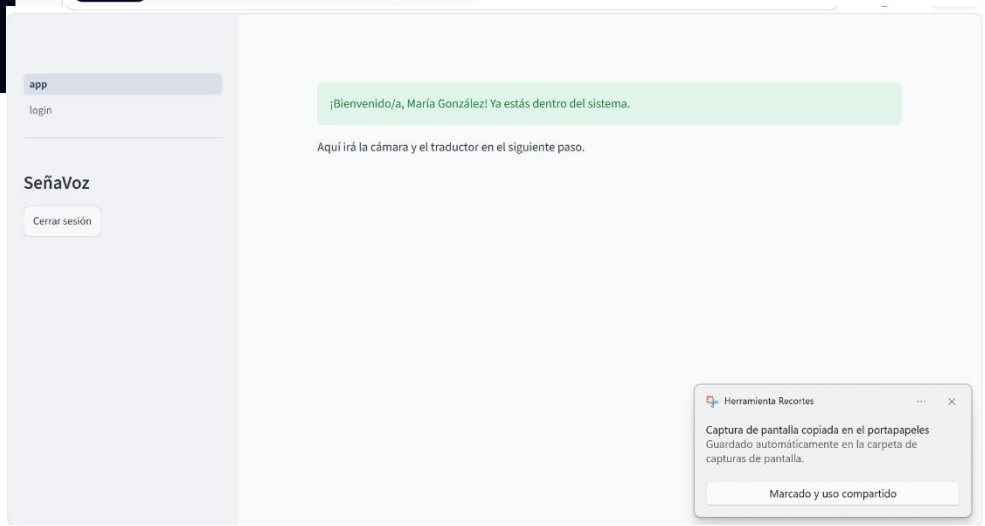
Capturas beta

En estas capturas se puede visualizar el como va mejorando la interfaz



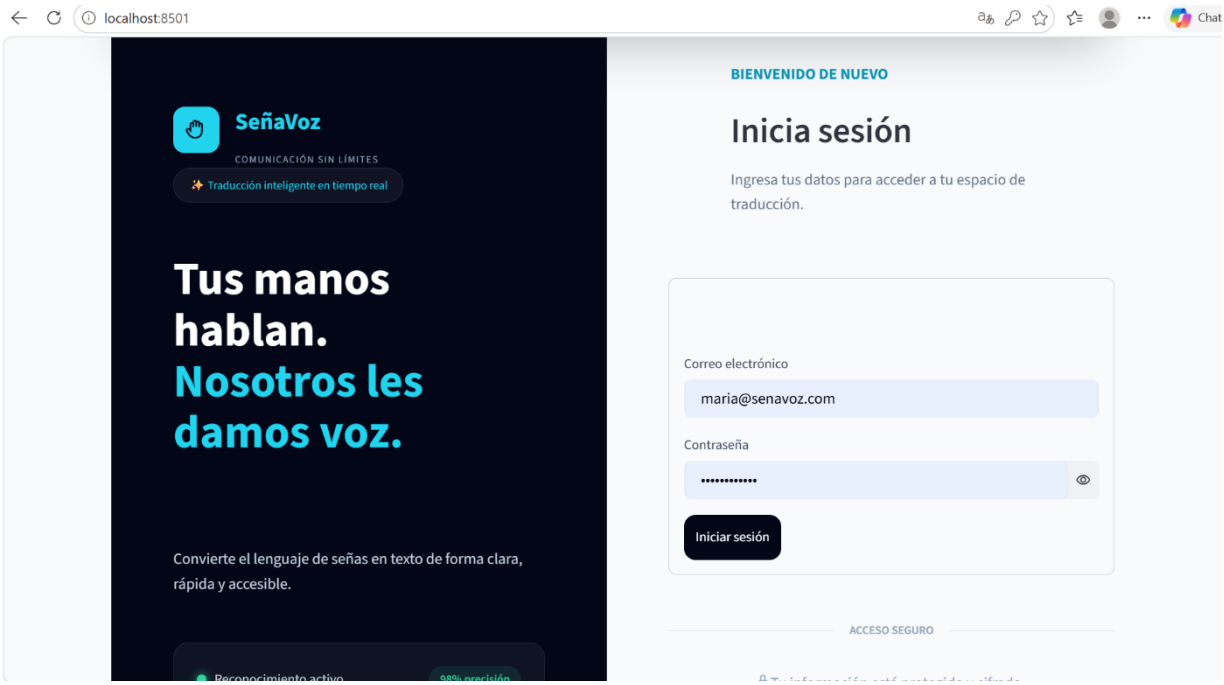




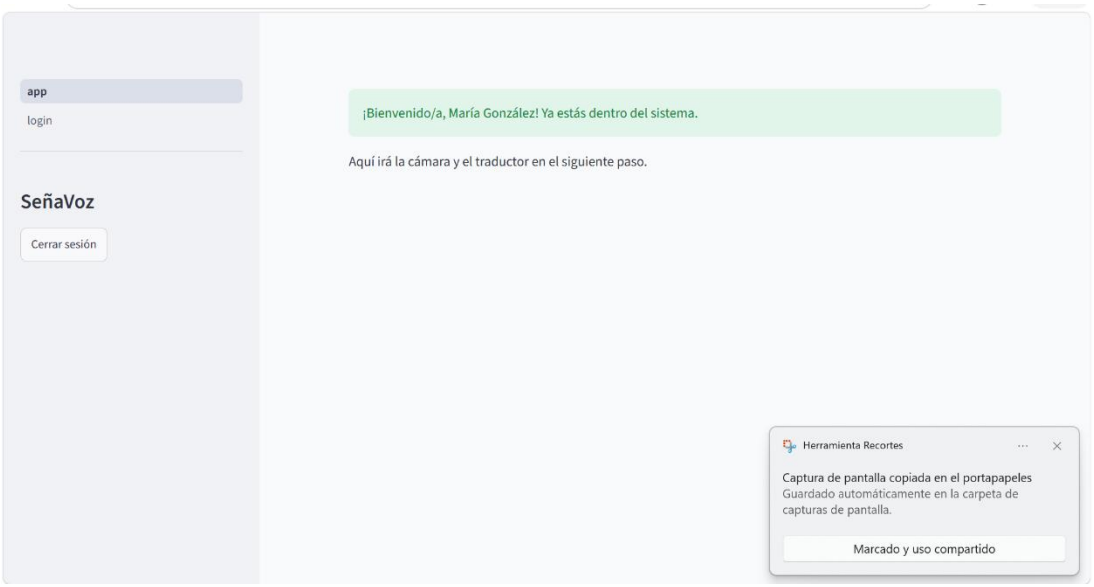


UX (Experiencia del usuario)

En este apartado se mostrará el cómo se va viendo la interfaz del usuario, aquí solo se muestra el inicio de sesión donde tienes que poner correo y contraseña para poder acceder a la aplicación.



En esta imagen se puede ver la otra interfaz donde ya estas dentro de la aplicación, pero en este apartado aun no se abre la cámara



Referencias biográficas

Guerrero-Balaguera, J. D., & Pérez-Holguín, W. J. (2015). Sistema traductor de la lengua de señas colombiana a texto basado en FPGA. Dyna, *82*(189), 172-181. https://doi.org/10.15446/dyna.v82n189.49635

Mejía Pérez, K., & Córdova Esparza, D. M. (2023). Reconocimiento de la Lengua de Señas Mexicana mediante redes neuronales. Revista Nthe, (41), 27-33. https://doi.org/10.29059/nthe.v0i41.172

Referencias del estado del arte:

Guerrero-Balaguera, J. D., & Pérez-Holguín, W. J. (2015). Sistema traductor de la lengua de señas colombiana a texto basado en FPGA. Dyna, 82(189), 172-181.

Mejía Pérez, K., & Córdova Esparza, D. M. (2023). Reconocimiento de la Lengua de Señas Mexicana mediante redes neuronales. Facultad de Informática, Campus Juriquilla, UAQ.

Modi et al. (2026). Real-Time American Sign Language Recognition via CNN-MediaPipe Integration. Zenodo.

Singh, A. K., Kumbhare, V. A., & Arthi, K. (2022). Real-time human pose detection and recognition using MediaPipe. Advances in Intelligent Systems and Computing, 145-154.

Tovar, L. A. (2011). La Importancia Del Estudio De Las Lenguas De Señas.

Zhang, F., et al. (2020). MediaPipe Hands: On-Device Real-Time Hand Tracking.

Rodríguez-González, M. A. (1992). Lenguaje de Signos. Confederación Nacional de Sordos de España.

Resumen del proyecto

La comunicación es un derecho básico, pero las personas con discapacidad auditiva enfrentan barreras al interactuar con la comunidad oyente, ya que la mayoría desconoce la Lengua de Señas Mexicana (LSM) y no siempre hay un intérprete disponible. Para reducir esta brecha, este proyecto desarrolla un traductor de señas del abecedario dactilológico a texto mediante visión por computadora y redes neuronales.

El sistema captura en tiempo real las señas con una cámara web, extrae los 21 puntos clave de la mano con MediaPipe y clasifica cada letra con una red neuronal multicapa (MLP) entrenada con un dataset propio. El resultado se muestra en una interfaz hecha con Streamlit, que presenta la letra detectada, su nivel de confianza y permite formar palabras letra por letra. La aplicación incluye inicio de sesión, modo de recolección de datos y se despliega en Docker para garantizar su reproducibilidad.

A diferencia de trabajos previos que requieren cámaras de profundidad (Mejía Pérez & Córdova Esparza, 2023) o hardware especializado como FPGA (Guerrero-Balaguera & Pérez-Holguín, 2015), esta propuesta funciona con una cámara web estándar, sin GPU y con herramientas de código abierto, lo que la hace accesible y de bajo costo. La meta es alcanzar una precisión de al menos 90 % en tiempo real. Como trabajo futuro se contemplan la síntesis de voz, el reconocimiento de señas dinámicas y de palabras completas.